FP 13 (Haskell 9): Making illegal states unrepresentable

(parse, don’t validate)

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

Aim of this lecture

By the end of this lecture you should:

- Know how why we define types that wrap other types
- Know how to use the type system to catch errors *before* they happen

Example 1: NASA Centimetres vs Inches catastrophe

In 1999, NASA lost the Mars Climate Orbiter because they **messed up their units**.

One part of the codebase used **inches** while the other used **centimeters.**

They **forgot to convert, since both were Ints**, and ended up basically assuming that 2 inches + 4 cm = 6 inches (obviously not true)

Wrapper types

We can fix this by defining wrapper types:

```
data Inches = In Int

data Centimetres = Cm Int

instance Num Inches where…

instance Num Centimetres where…
```

If we then try to `(In 1) + (Cm 2)`, we would get a type error. Looking at +:

`(+) :: Num a => a -> a -> a`, it needs both operands to be the **same type.**

Message state

Let’s extend this idea to modelling WhatsApp messages

```
data WhatsappMessage = Msg {
  content :: String,
  sender :: String,
  read :: Maybe Time,
  delivered :: Maybe Time,
  sent :: Maybe Time,
}
```

What would it mean for us to have a message to have a **read** time, but not a **sent** time?

Message state

```
data WhatsappMessage = Msg {
  content :: String,
  sender :: String,
  state :: MsgState
}

data MsgState = Draft
  | Sent { sent :: Time }
  |  Delivered { sent :: Time, delivered :: Time }
  | Read { sent :: Time, delivered :: Time, read :: Time }
```

Is it possible for us to construct a message without a send time but with a read time?

Nonempty list

We could also define a non-empty list:

```
data NonEmptyList = NonEmptyList a [a]
```

… which is just a list with an obligatory first element

Can we construct a `NonEmptyList` that has no elements?

Exceptions

Don’t let perfect be the enemy of good!

Go as far as you can with making illegal states unrepresentable, but going *overboard* can equally be detrimental.

It can harm interoperability, for one. Typeclasses implemented on Int may not be implemented on your custom PrimeInt data type.

[AR = “Fast and loose reasoning is morally correct” by Danielsson et al. Intro + 2nd section are fun and fairly familiar from Haskell with some googling]

Applications of these concepts

1. If we **parse** our data into types that enforce constraints, then work with the constrained version from then on, we only have to **validate once, at parse-time.** This prevents “shotgun-parsing”
2. We can **eliminate classes of logic errors,** preventing bugs before they happen: Inch + Cm is now a *type error.*
3. We can **move checks from runtime into compile type:** with our `NonEmptyList`, `head` can *never* crash at runtime.

Why Haskell here

Haskell's type system is **more expressive:** it lets us enforce **far more constraints** than other languages (e.g. it's awkward to define `MsgState` in Java

Honestly, just sum types are a game changer.

Exercises

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

- Define a type `data AtLeastOne a = ...` which can *either* have one value of `a` *or* have two (it CANNOT have zero, like `(Maybe a, Maybe a)`)
- Define the function `head :: NonEmptyList a -> a`
-

Reading

- https://lexi-lambda.github.io/blog/2019/11/05/parse-don-t-validate/
- https://functional-architecture.org/make_illegal_states_unrepresentable/